

FORMAN CHRISTIAN COLLEGE

(A CHARTERED UNIVERSITY)

COMPILER CONSTRUCTION

Programming Assignment 2

It's an open books and open notes assignment. Use of Internet is allowed. This assignment **MUST** be completed individually. You **CANNOT** share your code with any other course fellow. Any such attempt will be dealt with seriously.

Grading Criteria

Working Code: 80%

Properly formatted Report: 20%

Important: You need to submit a well formatted and well written report for this assignment.

The report format is given with this handout on page 7. Make sure to adhere to the format.

Start early. **NO** additional time in any case what so ever will be granted.

Viva will be conducted for this assignment, as well as previous assignment/s. Date will be announced later in class.

Hard Deadline: Soft copy of report along with the code and input file/s should be submitted on Moodle course page on or before Sunday May 05, 2024 before 11:59 pm.

A hard copy of the report MUST be submitted to the lab engineer on Monday May 06, 2024 in class.

Make sure to create a zip file of your submission and give it a name using given format:

COMP451_<your section>_<RollNumber>_Assgn-2

For example,

COMP451_X_12345678_Assgn2

Submissions through email will NOT be considered for grading.

Rubric

There are total of 12 instructions given. Your program must include at least six input files carrying all the 12 instructions in an appropriate manner. For example, each instruction must be included in at least two input files. Output should carry all three columns as shown in the sample run.

Exclusion of jump and branch instructions, but all other requirements are fulfilled: 50 out of 80

Exclusion of jump instruction, but all other requirements are fulfilled: 60 out of 80

All other requirements fulfilled but

Exclusion of address columns in the output will cost a deduction of 15 marks

Exclusion of instruction columns in the output will cost a deduction of 15 marks

Report will be graded based on the marks obtained in the coding section. For example, if someone obtains 60 marks out of 80 then the report will be graded out of 15 marks. Report marks are based on the quality of format and your writing ability.

Assignment Task [80 Marks]

In this task we will write an assembler. Your program should accept a text file from command line. The file should carry a valid and syntactically correct MIPS assembly program. For this assignment we assume that we have a limited number of MIPS assembly instructions. These instructions are:

- add
- and
- or
- andi
- ori
- xor
- addi
- sw
- lw
- slt
- beq
- j

Your program should read the input file line by line (or character by character) and translate each line in a valid MIPS machine code. The machine code should be displayed on console in hexa decimal form.

The complete format of the output is given in the sample runs.

Ignore the labels in your output as address of label is same as the address of instruction following it.

Make sure that your input programs must carry branch and jump instructions.

At least six input program files should accompany your final submission on MOODLE.

You may need to refer to the Internet to get yourself acquainted with the instruction formats of MIPS assembly language.

For your convenience, few links are given below:

https://www.dcc.fc.up.pt/~ricroc/aulas/1920/ac/apontamentos/P04_encoding_mips_instructions.pdf

<https://max.cs.kzoo.edu/cs230/Resources/MIPS/MachineXL/InstructionFormats.html>

<https://www.eecs.harvard.edu/~cs161/notes/mips-part-I.pdf>

https://www.dcc.fc.up.pt/~ricroc/aulas/1920/ac/apontamentos/P04_encoding_mips_instructions.pdf

Your task is to write a program in C that accepts a text file containing few lines (minimum three) of MIPS assembly program using the above instruction set. (It does not matter whether or not the program makes some useful output).

Note that you are not supposed to generate the output of the program. Your only intention is to generate the equivalent machine program. Don't worry if a program is incorrect semantically.

Note: We can provide any tailor-made program (using above instruction set only) to test your code.

Your program should first print the contents of the input file on console, and then the output should be displayed as shown in the sample runs.

Make sure to self-test the output of your program by assembling the input MIPS code in MARS and comparing the output with the output given in MARS.

COMP 451

The output of the program should be as shown. First column represents the address of the instruction. You can have this column hard coded starting with an address 0x00400000 and incrementing each line by 0x04.

Sample Run 1

```
$ gcc assgn2.c -o assgn
```

```
$ ./assgn inputFile
```

Assembly language program:

```
    addi    $10,$12,10
    sub     $15,$12,$10
    sw      $15,10($29)
```

It should then display the address, machine code and assembly instruction as output on the console as follows:

Machine Code:

0x00400000	0x218A000A	addi	\$10, \$12, 10
0x00400004	0x018A7822	sub	\$15, \$12, \$10
0x00400008	0xAFAF000A	sw	\$15, 10 (\$29)

COMP 451

Sample Run 2

```
$ gcc assgn2.c -o assgn
$ ./assgn inputFile
```

Assembly language program:

```
add    $9,$10,$11
addi   $12,$9,4
beq    $12,$9,end
lw     $11,16($10)
and    $13,$10,$12
end:
sw     $9,0($10)
```

It should then display the address, machine code and assembly instruction as output on the console as follows:

0x00400000	0x014B4820	add \$9,\$10,\$11
0x00400004	0X212C0004	addi \$12,\$9,4
0x00400008	0X11890002	beq \$12,\$9,end
0x0040000C	0x8D4B0010	lw \$11,16(\$10)
0x00400010	0x014C6824	and \$13,\$10,\$12
0x00400014	0xad490000	sw \$9,0(\$10)

COMP 451

Sample Run 3

```
$ gcc assgn2.c -o assgn
$ ./assgn inputFile
```

Assembly language program:

```
start:
beq    $t5,$t6,end
add     $t1,$t2,$t3
andi    $t4,$t1,4
xor     $t3,$t2,$t8
slt     $t6,$t4,$t7
andi    $t6,$t6,1
j       start
end:
sw      $t1,($t2)
```

It should then display the address, machine code and assembly instruction as output on the console as follows:

0x00400000	0x11AE0006	beq \$t5,\$t6,end
0x00400004	0x014B4820	add \$t1,\$t2,\$t3
0x00400008	0x312C0004	andi \$t4,\$t1,4
0x0040000C	0x01585826	xor \$t3,\$t2,\$t8
0x00400010	0x018F702A	slt \$t6,\$t4,\$t7
0x00400014	0x31CE0001	andi \$t6,\$t6,1
0x00400018	0x08100000	j start
0x0040001C	0Xad490000	sw \$t1,(\$t2)

COMP 451

Self Test Procedure

Each input file should be augmented with an output snapshot file which should carry a snapshot from the MARS IDE for the corresponding input file. A sample input file and its corresponding snapshot file is shown below:

Input file-3:

```
start:
beq    $t5,$t6,end
add    $t1,$t2,$t3
andi   $t4,$t1,4
xor     $t3,$t2,$t8
slt     $t6,$t4,$t7
andi   $t6,$t6,1
j      start
end:
sw     $t1,($t2)
```

MARS Snap Shot for Input file-3

Text Segment					
Bkpt	Address	Code	Basic		
☐	0x00400000	0x11ae0006	beq \$13,\$14,0x00000006	5:	beq \$t5,\$t6,end
☐	0x00400004	0x014b4820	add \$9,\$10,\$11	6:	add \$t1,\$t2,\$t3
☐	0x00400008	0x312c0004	andi \$12,\$9,0x00000004	7:	andi \$t4,\$t1,4
☐	0x0040000c	0x01585826	xor \$11,\$10,\$24	8:	xor \$t3,\$t2,\$t8
☐	0x00400010	0x018f702a	slt \$14,\$12,\$15	9:	slt \$t6,\$t4,\$t7
☐	0x00400014	0x31ce0001	andi \$14,\$14,0x0000...	10:	andi \$t6,\$t6,1
☐	0x00400018	0x08100000	j 0x00400000	11:	j start
☐	0x0040001c	0xad490000	sw \$9,0x00000000(\$10)	13:	sw \$t1,(\$t2)

To conclude, your submission should carry following files:

- Your code file/s. Should be named as <your name>.asm
- Your input files. Should be named as in1, in2, in3, in4, in5, and in6
- The corresponding MARS snapshot files. Should be named as snap1, snap2, snap3, snap4, snap5, and snap6. Note that the output of your program for a specific input file should conform to the corresponding snap shot file.
- The soft copy of the report according to the new format.

DONOT forget to submit a hardcopy of your report to the lab engineer on prescribed date and time.

COMP 451

Assignment 2 COMP 451 Section X

Muhammad Rauf Butt 12345678

Format Christian College (A Chartered University)

raufbutt@fccollege.edu.pk

Abstract

A short (usually around 250-400 words) description of the paper. Should include what the purpose of the paper is (including the basic research question/problem), the basic design of your project, and the major findings

Introduction

A general introduction to your topic and what you expect to learn from your project or experiment. Your research question should be found here.

Literature Review

An analysis of what has already been published about your chosen topic. Should be able to show how your research question fits into the context of your field.

Methods

A description of everything you did in your experiment or project, step-by-step. Needs to be detailed enough so that any **reader would be able to repeat each step exactly on their own.**

Results

What actually happened during your project or what you found at the end of your experiment. This is usually the best part to include the majority of your graphs, photos, tables, and other visual aids, as long as they help explain the results of your work.

Discussion

An analysis of the results that integrates what you found into the wider body of research in your field. Can also include future hypotheses to be tested or future projects to build from your own.

Conclusions

Can be included in the discussion if necessary. A final summary of the paper, including whether or not you were able to answer your original research question.

References

The reference page(s) is a list of all the sources you used to research and create your project/experiment, including everything cited in the literature review and methods sections. Remember to use the same citation style throughout the paper. An appendix would include any additional information about your work that you were not able to include within the body of your paper (like large datasets and figures) that would help readers better understand your results.